

Inleiding

Threat modeling is een systematische techniek om mogelijke dreigingen tegen een softwaresysteem te identificeren, te analyseren en te prioriteren nog vóór implementatie of tijdens evolutief onderhoud. In essentie dwingt threat modeling ontwikkelaars en architecten om expliciet na te denken over “wat kan er misgaan?”, “wat is de impact?” en “welke maatregelen zijn nodig?”, en vormt het daarmee een kernonderdeel van secure software engineering. Het proces omvat typisch het in kaart brengen van systeemcomponenten en dataflows, het identificeren van potentiële aanvallers en aanvalspaden, en het koppelen van passende mitigaties aan de belangrijkste risico's.

Binnen de softwareontwikkelingscyclus kan threat modeling op verschillende momenten worden ingezet, bijvoorbeeld tijdens requirementsanalyse, architectuurontwerp of bij grote refactorings. In een ideale DevSecOps-context wordt threat modeling iteratief uitgevoerd, parallel aan sprints, zodat nieuwe features meteen op beveiligingsimpact worden beoordeeld. In de praktijk blijkt dit echter moeilijk vol te houden: tijdsdruk, beperkte securityexpertise en een gebrek aan passende tooling zorgen ervoor dat threat modeling vaak wordt uitgesteld, oppervlakkig uitgevoerd of volledig overgeslagen.

Door de jaren heen zijn diverse methoden ontwikkeld om threat modeling te structureren. Een bekende categorie zijn taxonomie-gebaseerde benaderingen, zoals STRIDE, die dreigingen klasseren op basis van hun effect (bijvoorbeeld spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege). Andere benaderingen maken gebruik van attack trees of misuse cases, waarin ongewenst gedrag hiërarchisch wordt opgesplitst in deelstappen en varianten. In meer industriële context worden deze methoden vaak ondersteund door generieke diagramtools of mindmapsoftware, die wel flexibiliteit bieden maar weinig domeinspecifieke ondersteuning voor prioritering en statusopvolging.

Voor ontwikkelaars zonder diepgaande securityachtergrond vorm deze klassieke methoden een aanzienlijke drempel: ze vergen vertrouwdheid met beveiligingsconcepten, formele notaties en lijsten van standaarddreigingen, terwijl de beschikbare tools zelden gericht zijn op gebruiksgemak of leerbaarheid. Hierdoor dreigt threat modeling een specialistische activiteit te blijven, in plaats van een breed gedragen ontwerpraktijk binnen ontwikkelteams. Deze kloof tussen theoretisch aanbevolen werkwijzen en feitelijke toepassing in de praktijk vormt een belangrijk motief voor onderzoek naar meer toegankelijke, visueel ondersteunde en gebruiksvriendelijke threat-modelingbenaderingen

Dit onderzoek speelt in op deze drempels door een gebruiksvriendelijke webtool te ontwerpen en evalueren, gebaseerd op de Fortunately–Unfortunately-methode, waarin dreigingen en tegenmaatregelen in een intuïtieve boomstructuur worden voorgesteld. De

tool integreert specifieke UX-features zoals kleurcodering, statusiconen, danger rating, branch summary en filtering om zowel navigatie als prioritering te ondersteunen en wordt geëvalueerd in een gebruikersonderzoek waarin de tool wordt vergeleken met een meer generieke referentiesoftware. Het doel van de masterproef is na te gaan in welke mate deze combinatie van visuele representatie en UX-features in combinatie met de Fortunately–Unfortunately-methode, threat modeling toegankelijker, efficiënter en effectiever maakt voor niet-security-experts.

Het belang van dit onderzoek ligt enerzijds in het verlagen van de instapdrempel voor threat modeling, zodat ook ontwikkelaars zonder uitgesproken securityprofiel beter onderbouwde risicoanalyses kunnen uitvoeren. Anderzijds draagt het werk bij aan HCI-onderzoek rond gebruiksvriendelijke security-tools door empirisch te onderzoeken welke interface-elementen (zoals kleuren, iconen, filters en tooltips) daadwerkelijk bijdragen aan taakprestatie, gebruikservaring en feature-voorkeur. Daarmee sluit de masterproef aan bij een bredere verschuiving van puur technische security-oplossingen naar mensgerichte, visueel ondersteunde hulpmiddelen.

De rest van deze masterproef is als volgt opgebouwd. Hoofdstuk 2 bespreekt de theoretische achtergrond en gerelateerd werk rond threat modeling, de Fortunately–Unfortunately-methode en usability- en UX-evaluatie in securitycontexten. Hoofdstuk 3 beschrijft het ontwerp en de implementatie van het React/D3-webprototype en licht het onderzoeksdesign van de gebruikerstudie toe, inclusief taken, meetinstrumenten en analysemethoden. Hoofdstuk 4 presenteert de empirische resultaten over taakprestaties, user experience en feature-ranking, gevolgd door een discussie in hoofdstuk 5 waarin deze bevindingen worden geïnterpreteerd, gekoppeld aan de literatuur en aangevuld met beperkingen en implicaties voor praktijk en toekomstig onderzoek. Hoofdstuk 6 besluit met de belangrijkste conclusies en schetst mogelijke uitbreidingen, zoals AI-ondersteuning en collaboratieve functies voor breder gebruik in onderwijs en industrie.

Hoofdstuk 2: Gerelateerd werk

Hoofdstuk 3: Ontwerp en implementatie

Ontwerpdoelstellingen

Het ontwerp van de webtool vertrekt vanuit de nood om threat modeling concreet uitvoerbaar te maken voor ontwikkelaars zonder uitgebreide securityachtergrond, met een sterke focus op leesbaarheid van de boom, directe visuele feedback en eenvoudige navigatie. De codebasis is daarom opgezet rond een beperkt aantal, duidelijk herkenbare visuele bouwblokken (nodes, iconen, panelen) die in de volledige interface consequent worden hergebruikt. De Fortunately–Unfortunately-structuur wordt afgedwongen op code-niveau door het node-type expliciet op te slaan en in de renderlogica te gebruiken om kleuren, iconen en interactie te bepalen.

Architectuur en technologie

De tool is geïmplementeerd als een single-page webapplicatie met React en next.js voor de componenthiërarchie en D3-functionaliteit voor de boomlay-out en interactie. De kern van de applicatie bestaat uit:

- een datastructuur voor nodes (name, children, danger rating, level, status, statusColor);
- een visualisatielaag met React-componenten voor nodes, iconen, zijpanelen en tooltips;
- helperfuncties om paden naar de root te berekenen (voor branch summary) en gefilterde lijsten op basis van status en danger rating samen te stellen.

De data worden in een boomstructuur bijgehouden, terwijl D3 wordt gebruikt om posities te berekenen en transities (uitklappen, inklappen) te ondersteunen. React zorgt ervoor dat elke wijziging in de onderliggende data (level, status, statusColor) onmiddellijk leidt tot een hertekening van de corresponderende componenten.

Niet-geïmplementeerde alternatieven

Circulaire tree maps en treemaps werden theoretisch geëvalueerd aan de hand van bestaande voorbeelden en literatuur over hiërarchische visualisatie. Circulaire lay-outs, waarin nodes rond een centrale root worden gerangschikt met unfortunately-nodes prominent op basis van danger rating, bleken ongeschikt omdat sequentiële fortunately/unfortunately-relaties visueel fragmentarisch worden en parent-child-paden cognitief moeilijk traceerbaar blijven. Treemaps, met unfortunately-nodes als dominante blokken en fortunately-nodes als subregio's geschaald op danger rating, prioriteren risico-overzicht maar schenden de lineaire causaliteitslogica van de methode en verhogen de cognitieve belasting bij diepere bomen. Op basis van deze analyse werden deze technieken niet geïmplementeerd.

Geïmplementeerde en afgewezen lay-outs

Horizontale en verticale boomlay-outs werden concreet geïmplementeerd en intern geëvalueerd. De horizontale variant visualiseert fortunately/unfortunately-alternering van links naar rechts, terwijl de verticale variant een top-down hiërarchie met uitklapbare takken volgt. Beide implementaties gebruikten dezelfde node-componenten en UX-features, maar resulteerden in suboptimale prestaties: excessieve witruimteverbruik, verminderde leesbaarheid van lange sequenties en ontoereikende ondersteuning voor zowel globale oriëntatie als lokale detailanalyse. Deze lay-outs werden daarom verworpen ten gunste van een alternatieve fileview-representatie.

Gekozen fileview-visualisatie

De definitieve visualisatie maakt gebruik van een fileview-geïnspireerde lay-out, waarin de boom als een hiërarchische bestandsstructuur wordt voorgesteld met consistente node-weergave en minimale witruimte. Deze keuze balanceert sequentiële leesbaarheid met schaalbaarheid en werd exclusief gebruikt tijdens de gebruiker studie.

Node-representatie in de code

De Node-component ontvangt minstens twee props: de naam (beschrijving) en het niveau/level (“fortunately” of “unfortunately”). In de renderfunctie wordt eerst het type genormaliseerd (via `level.toLowerCase()`), waarna een conditionele tak de juiste stijl toepast:

- voor “fortunately” wordt een groene achtergrond met afgeronde hoeken, zwarte rand en center-uitlijning gerenderd;
- voor “unfortunately” wordt een rode variant met dezelfde afmetingen en typografie gebruikt.

Door alle nodes via dezelfde component te laten lopen, zijn zaken als minimale en maximale breedte, tekstomloop, padding en border overal identiek, wat de leesbaarheid van langere ketens bevordert. Boven elke node wordt in kleinere typografie bovendien het corresponderende label “*fortunately*” of “*unfortunately*” weergegeven, zodat de betekenis van de kleurcodering ook tekstueel ondersteund wordt. De kleurcodering en label komen dus rechtstreeks uit de componentlogica, niet uit losse inline-stijlen verspreid door de code.

Statusiconen en danger rating

Statusiconen worden in aparte, herbruikbare componenten ondergebracht (een `IconSelectorButton` of een `StatusIcon`-component) die op basis van een statusveld in de node-data het juiste icoon en de bijhorende kleur kiezen. Typische statussen zijn Weakness (rood schild), Weakness Under Review (oranje schild), Assumption Under

Review (oranje checkschild) en Verified Assumption (groen checkschild). Als extra aanduiding is er een blauw checkschild beschikbaar voor nodes die volledig zijn afgewerkt (bij fortunately-nodes) of als *low importance* worden beschouwd (bij unfortunately-nodes). Deze twee statussen worden niet gebruikt in de filterfunctionaliteit, maar bieden gebruikers een bijkomende mogelijkheid om tussenliggende of minder belangrijke nodes te labelen en zo hun status expliciet te communiceren binnen het team. De iconcomponent wordt in de node-render opgenomen, zodat elke node automatisch het juiste symbool toont.

De danger rating wordt als getal in de node-data opgeslagen, berekend uit likelihood en impact (twee schaalwaarden van 1 tot 5). In de code wordt deze rating gebruikt om nodes in de filterweergave aflopend te sorteren (hoogste rating bovenaan) en permanent bij elke unfortunately-node weer te geven, zowel in de boom als in de filterlijsten zelf. Dit contrasteert met secundaire acties zoals add, delete en collapse die pas bij hover verschijnen, waardoor de interface visueel niet wordt overbelast en de focus op de essentie van de data blijft. Deze constante zichtbaarheid is cruciaal voor prioriteren van weaknesses, snelle risicotriage tussen nodes en overzicht van het totale risicolandschap. Door centrale berekening en sortering kan de rating consistent in alle weergaven gebruikt worden.

Door de berekening en sortering centraal te houden, kan de rating zowel in de boomvisualisatie als in lijsten op een consistente manier gebruikt worden.

Branch summary en filterlogica

De branch summary en filterfunctionaliteit zijn geïmplementeerd als aparte zijpanelen en ondersteunende hulpfuncties, die de fileview-navigatie faciliteren door globale context en taakgerichte prioritering te bieden.

De branch summary reconstrueert het hiërarchische pad van de rootnode tot de geselecteerde node via parent-relaties. Hiertoe worden algoritmes zoals `getImmediateParent` en `getParentsAbove` aangewend om bovenliggende nodes recursief op te bouwen. Het resulterende pad wordt samenvoegt tot een compacte tekstuele en visuele samenvatting, gepresenteerd in een zijpaneel weergave die voor optimale leesbaarheid zorgt. Selectie van een gefilterde node triggert een state-update met de corresponderende node-ID, wat in de Node-component resulteert in visuele highlighting (bijv. border- of kleurverandering) en scrolling naar de relevante positie in de fileview.

De filterfunctionaliteit groepeert nodes op statuscategorie (bijv. Weakness, Weakness Under Review, Assumption Under Review, Verified Assumption) met behulp van een map-structuur, waarbij index in de map als sleutel fungeert. Binnen weakness-categorieën vindt sortering plaats op danger rating (dalend), zodat kritieke items vooraanstaand worden gepositioneerd. Ook in dit zijpaneel resulteert een nodeklik in

een visuele highlighting (bijv. border- of kleurverandering) en scrolling naar de relevante positie in de boom.

Deze implementatie verbindt de abstracte datastructuur (statussen, danger ratings, parent-relaties) naadloos met concrete interface-interacties, waardoor gebruikers zowel lokale detailanalyse als globale oriëntatie kunnen uitvoeren zonder significante contextverlies. Naast de visuele kleurcodering van nodes wordt bovendien een letterlabel toegevoegd, F (Fortunately) of U (Unfortunately), dat semantisch aangeeft of de node een positief dan wel negatief geëvalueerde uitkomst vertegenwoordigt. Dit bevordert snelle interpretatie, vooral bij dense weergaven of wanneer kleuronderscheid minder prominent is.

Hover-functionaliteit voor parent-context

In zowel de boomvisualisatie als het filterzijpaneel wordt bij hover over een node de directe parent getoond, zodat de hiërarchische context expliciet zichtbaar blijft tijdens interactie. In de boom ondersteunt dit snelle oriëntatie bij complexe structuren zonder terug scrollen, terwijl het in het filterzijpaneel gebruikers toelaat gefilterde nodes (bijv. alle Weaknesses) te inspecteren zonder de volledige boompositie te traceren. De parent-weergave voorkomt dat kritieke relaties verloren gaan in een lijstcontext en biedt snelle contextuele inspectie vóór navigatie (bijv. "welke fortunately-node adresseert deze weakness?").

Ondersteuning en feedback in de UI

In de implementatie is ondersteuning vooral uitgewerkt met hover-tooltips en consistente iconografie. Iconen en statussen krijgen een tooltipcomponent die bij hover de betekenis van het symbool, de status of de rating verklaart, gebruikmakend van korte, vaste teksten. Node-selectie, statuswijzigingen en filteracties geven directe visuele feedback via kleuraccenten, borders of highlight-effecten, zodat de gebruiker zonder aparte meldingen begrijpt wat er in de boom is veranderd.

Toepassing van Nielsen's usability-heuristieken in de webtool

Het ontwerp en de implementatie van de fortunately-unfortunately-webtool zijn expliciet afgestemd op de tien usability-heuristieken van Nielsen. De onderliggende React-componenten (Page, TreeVisualizer, SearchBar, IconSelectorButton) realiseren deze heuristieken concreet in de gebruikersinterface.

1. Zichtbaarheid van de systeemstatus

De tool geeft gebruikers voortdurend feedback over de huidige toestand van het systeem. De boomweergave in TreeVisualizer wordt onmiddellijk geüpdatet na toevoeging, bewerking of verwijdering van nodes via `setTreeData`. De geselecteerde knoop (`currentNode`) en zoekresultaten (`highlightedNodes`) worden visueel geaccentueerd met een extra rand en achtergrondkleur. De

shield-knoppen met teller in de rechterzijbalk ([redNodes.length], [greenNodes.length] enz.) tonen onmiddellijk na gebruikersinteractie hoeveel zwaktes en aannames zich in elke status bevinden. De “Danger rating”-indicator per unfortunately-knoop (component NodeDangerPopup) geeft continu de actuele risicoscore weer, die direct wijzigt bij aanpassing van impact en likelihood.

2. **Overeenkomst tussen systeem en de echte wereld**

De webtool gebruikt terminologie en structuren die aansluiten bij gangbare praktijken in security- en risicoanalyse. Nodes zijn expliciet getypeerd als level = “fortunately” of “unfortunately”, wat overeenkomt met positieve en negatieve implicaties in het analyseproces. De rechterzijbalk groepeerde nodes in domeinspecifieke categorieën: “Weaknesses”, “Weaknesses Under Review”, “Assumptions Under Review” en “Verified assumptions”, ondersteund door passende schild-iconen (GoShield, GoShieldCheck). De risicoscore wordt opgebouwd uit de dimensies “Impact” en “Likelihood” en weergegeven als “Danger rating”, wat aansluit bij gangbare risico-taxonomieën.

3. **Gebruikerscontrole en vrijheid**

De implementatie voorziet diverse mechanismen die de gebruiker controle geven over het modelleringsproces. Via “Reset Tree” kan op elk moment naar een lege beginsituatie worden teruggekeerd; deze actie wordt voorafgegaan door een bevestigingsdialoog (window.confirm) om onbedoeld dataverlies te vermijden. Op knooppniveau kunnen gebruikers afzonderlijke elementen verwijderen via de prullenbakknop (GoTrash) of nieuwe kinderen toevoegen (RiAddBoxLine). Panelen zoals de rechter “shields”-sidebar (isOpen) en het linker “Parents above current node”-overzicht (openParentSummary) kunnen naar believen worden geopend en gesloten, zodat gebruikers vrij kunnen schakelen tussen een compacte en een meer gedetailleerde weergave.

4. **Consistentie en standaarden**

In zowel de code als de visuele vormgeving is een consistente logica doorgevoerd. Fortunately-knooppunten worden systematisch in groen weergegeven, unfortunately-knooppunten in rood (component Node en de achtergrondklassen in ShowNodes), een kleurcodering die ook terugkomt in de zijbalk (groene/rode labels met “F” en “U”) en in de badges voor gevaar (FiTriangle). De betekenis van iconen blijft overal gelijk: GoShield markeert zwaktes, GoShieldCheck markeert aannames of voltooide nodes, GoInfo toont branchesamenvatting, en GoChevronDown/Up staan voor uit- en inklappen. Interactie patronen zoals hover-tooltips, modale bevestigingen en scrollIntoView sluiten aan bij typische webstandaarden, zodat gebruikers bestaande ervaring met andere applicaties kunnen hergebruiken.

5. Foutpreventie

Het ontwerp tracht fouten te voorkomen door de interactieruimte te beperken tot zinvolle acties. Bij het toevoegen van een kindknoop bepaalt de applicatie automatisch het type (fortunately of unfortunately) op basis van het level van de ouder, waardoor semantisch inconsistente ketens vermeden worden.

Destructieve acties zoals het verwijderen van een knoop of het resetten van de volledige boom vereisen expliciete bevestiging via `window.confirm`. Dropdowns voor impact en likelihood zijn begrensd tot waarden 1–5, wat foutieve invoer uitsluit en tegelijk een uniforme schaal afdwingt. Klikken buiten pop-ups (zoals het danger-ratingvenster of de iconselector) sluit deze vensters automatisch, waardoor de kans op onbedoelde verdere interactie in een oude context afneemt.

6. Herkenning boven herinnering

De interface is zo ontworpen dat relevante acties en informatie zichtbaar worden aangeboden in plaats van uit het geheugen te moeten worden opgeroepen. Contextuele knoppen rond elke knoop (iconselector, expand/collapse, delete, add child) verschijnen bij hover, zodat gebruikers zien welke handelingen mogelijk zijn zonder menu's of commando's te onthouden. De `IconSelectorButton` toont bij hover een korte tekstuele uitleg ("Weakness", "Assumption Under Review", "Verified assumption", enz.), waardoor de betekenis van de iconen direct herkenbaar is. De `SearchBar` laat gebruikers zoeken op knoopnaam; gevonden nodes worden automatisch gehighlight en centraal in beeld gescrold (`scrollIntoView`), waardoor de gebruiker de exacte positie in de boomstructuur niet hoeft te onthouden.

7. Flexibiliteit en efficiëntie van gebruik

De tool ondersteunt zowel exploratief gebruik door beginners als efficiëntere workflows voor ervaren gebruikers. Beginners kunnen via de knoppen in de boom en de zijbalk lineair nodes toevoegen en labels aanpassen. Tegelijk kunnen ervaren gebruikers grote modellen snel manipuleren via globale collapse/expand-functionaliteit (`CollapseButtons`) en door herhaaldelijk op Enter te drukken in het zoekveld om cyclisch door alle matches te navigeren (`highlightIndex`). De dynamische lijsten van zwaktes en aannames in de rechterzijbalk fungeren als werkvoorraad: een klik op een item zorgt ervoor dat de relevante tak automatisch wordt uitgekapt en in beeld komt, wat gericht en efficiënt werken bij complexe bomen ondersteunt.

8. Esthetisch en minimalistisch ontwerp

De interface combineert een eenvoudige lay-out met terughoudende visuele elementen. De centrale boomweergave is dominant en gebruikt een beperkt kleurenpalet (groen/rood/grijs) om de belangrijkste semantische verschillen te markeren. Secundaire acties verschijnen pas bij hover (`group-hover:opacity-100`),

waardoor de standaardweergave relatief schoon blijft. De rechterzijbalk is standaard samengevouwen tot compacte “shield”-knoppen; het volledige overzichtspaneel wordt alleen getoond wanneer de gebruiker hier expliciet voor kiest. Tooltips zijn kort en beperkt tot contextuele uitleg, zodat er geen overdadige tekst of iconografie ontstaat die de cognitieve belasting zou verhogen.

9. Helpen bij herkennen, diagnosticeren en herstellen van fouten

Hoewel geavanceerde foutafhandeling nog beperkt is, ondersteunt de tool gebruikers bij het herkennen en corrigeren van fouten. Bevestigingsdialogen bij verwijderen en resetten maken expliciet welke actie op het punt staat uitgevoerd te worden, zodat de gebruiker een vergissing kan terugdraaien door te annuleren. In de NodeDangerPopup worden de actuele danger rating, impact en likelihood expliciet getoond, waardoor incongruente waarden makkelijk opgemerkt en aangepast kunnen worden. De branch-samenvatting (via de GoInfo-knop en getParentsAbove) helpt gebruikers bij het diagnosticeren van structurele fouten in de redeneringsketen: door de context van een knoop te tonen, wordt het eenvoudiger vast te stellen of een veronderstelling op de juiste plaats staat.

10. Help en documentatie

Tot slot voorziet de webtool in lichte, geïntegreerde ondersteuning. Verschillende elementen bevatten inline-tooltips die bij hover verschijnen, zoals “Danger rating” bij de rode driehoek, “Show branch summary” bij het infopictogram en semantische labels bij de icons in IconSelectorButton (“Weakness Under Review”, “Completed node”, “Low Importance”). De zoekbalk is duidelijk gelabeld (“Search for node”) en gebruikt een herkenbare Material-UI TextField, waardoor de functie intuïtief is. Deze vorm van embedded documentatie reduceert de noodzaak voor een omvangrijke externe handleiding en ondersteunt vooral eerste-gebruikssessies.

Prototypebeperkingen

De huidige codebasis is opgezet als proof-of-concept:

- opslag gebeurt lokaal en er is nog geen integratie met een backend of externe threat-databanken;
- er is geen multi-user synchronisatie, waardoor collaboratieve workflows buiten scope vallen;
- performance-optimalisaties voor extreem grote bomen zijn beperkt tot basismaatregelen (zoals uitklapbare takken).

Deze keuzes houden de implementatie beheersbaar en laten toe om de impact van de visualisatie- en UX-beslissingen in de gebruikerstudie te evalueren, zonder afgeleid te worden door infrastructuur of integraties.

Gebruikersstudie

Verloop van de gebruikersstudie

De studie had als doel de bruikbaarheid, duidelijkheid en efficiëntie van de dreigingsmodelleringstool te evalueren bij het analyseren aan de hand van 2 scenario's, namelijk het uitwerken van een securitymodel voor een inlog- en authenticatiesysteem als scenario één en het analyseren van een AI fraude detectiesysteem. Deelnemers voerden gestructureerde scenario-taken uit in de software en vulden nadien een User Experience Questionnaire (UEQ) en open vragen in.

Deelnemers kregen eerst een introductie tot de fortunately–unfortunately-methode en de bijkomende statussen (Weakness, Weakness Under Review, Assumption Under Review, Verified Assumption) en danger rating. Vervolgens doorliepen ze de twee scenariu's, elk gekoppeld aan één van de twee softwareversies.

Softwareversies

De basisversie van de tool bevatte enkel kernfunctionaliteit: nodes toevoegen, nodes verwijderen, alle nodes inklappen, alle nodes uitklappen, per node inklappen en een zoekfunctie. In deze versie moesten de status en soort van een node (fortunately/unfortunately, inclusief statuslabels zoals Weakness of Verified Assumption) expliciet in de node-naam zelf worden opgenomen door de gebruiker.

De uitgebreide versie bouwde hierop voort met bijkomende visualisatie- en analysefeatures, zoals kleurcodering voor het soort node, iconen voor classificatie, filtering op status, telling van nodes per status, rangschikking op basis van danger rating, highlighting vanuit de filterbalk, samenvattingsfunctionaliteit en tooltips. Beide versies gebruikten dezelfde onderliggende fileview tree-representatie van het fortunately–unfortunately-model, geïmplementeerd in de codebasis van de prototypeapplicatie.

Taken en procedure

Per scenario kregen deelnemers een gedeeltelijk ingevuld model en moesten zij dit verder verkennen, analyseren en aanvullen via vier opeenvolgende taken. Typische subtaken waren: ontbrekende unfortunately-nodes toevoegen, nieuwe fortunately-tegenmaatregelen uitwerken, danger ratings invullen, statussen correct toekennen en lijstjes van specifieke nodecategorieën opstellen (bijvoorbeeld alle weakness-nodes of de weakness under review met hoogste danger rating).

Daarnaast voerden deelnemers taken uit om informatie uit de tree te halen, zoals het opzoeken van alle Verified Assumption-nodes of het reconstrueren van de keten van nodes van de root tot een specifieke node. In de laatste taak van elke story moest de deelnemer een samenvatting opstellen van de volledige fortunately–unfortunately-keten

tot aan een gekozen of gegeven node, als maatstaf voor de ondersteuning van de tool bij communicatie en documentatie.

Dataverzameling

Tijdens de sessie werd het scherm van de deelnemer opgenomen en live gevolgd, zodat interactiepatronen en fouten later kwalitatief konden worden geanalyseerd. Na elke versie vulden deelnemers de UEQ in en beantwoordden ze open vragen over welke features nuttig waren, welke verbeterd konden worden en welke versie ze in een reële werksituatie zouden verkiezen.

De studie verzamelde zo zowel kwantitatieve gebruikservaringsscores als kwalitatieve feedback over de toegevoegde visualisatiefeatures ten opzichte van de basisfunctionaliteit. Alle gegevens werden gepseudonimiseerd en enkel gebruikt voor onderzoeksdoeleinden, conform de informed-consentprocedure.

User Experience Questionnaire

De User Experience Questionnaire (UEQ) is een gevalideerde, semantisch-differentiaal gestandaardiseerde vragenlijst die een holistische beoordeling van de gebruikerservaring mogelijk maakt door middel van zes schalen: aantrekkelijkheid (algemene indruk van het product), duidelijkheid (leercurve en vertrouwd raken), efficiëntie (taakuitvoering zonder onnodige inspanning en reactietijd), betrouwbaarheid (gevoel van controle, veiligheid en voorspelbaarheid), stimulatie (opwinding, motivatie en plezier) en nieuwheid (creativiteit en interesseopwekkend ontwerp).

Deze schalen omvatten zowel pragmatische bruikbaarheidsaspecten als hedonistische gebruikservaringselementen, en de vragenlijst is beschikbaar in meer dan 30 talen, vergezeld van gratis hulpmiddelen zoals PDF-versies, Excel-analysebladen en een verkorte UEQ-S-versie met acht items voor specifieke scenario's.

Beschrijving van de UEQ-Schalen

De UEQ meet gebruikerservaring via gepaarde tegenstellingen op een 7-punts Likert-schaal, waarbij hogere scores positieve ervaringen aangeven.

- **Aantrekkelijkheid:** Beoordeelt de algemene waardering van het product door gebruikers.
- **Duidelijkheid:** Evalueert de eenvoud waarmee gebruikers vertrouwd raken en het product leren gebruiken.
- **Efficiëntie:** Analyseert de mogelijkheid tot moeiteloze taakoplossing en snelle reactietijden.
- **Betrouwbaarheid:** Onderzoekt het gevoel van controle, voorspelbaarheid en veiligheid tijdens interacties.

- **Stimulatie:** Meet de mate waarin het product motiverend, opwindend en plezierig is.
- **Nieuwheid:** Beoordeelt de creativiteit van het ontwerp en de interesse die het opwekt.

Relevantie voor dit onderzoek

De UEQ biedt een robuuste, kwantitatieve methode voor het evalueren van gebruikersinterfaces in threat modeling-tools, door pragmatische schalen (duidelijkheid, efficiëntie, betrouwbaarheid) te combineren met hedonistische aspecten (stimulatie, nieuwheid), wat directe vergelijkingen tussen prototypes, softwareversies of benchmarks mogelijk maakt.

In gebruikersstudies fungeert de UEQ als complementaire tool voor kwalitatieve methoden, ondersteund door eenvoudige statistische analyses via bijgeleverde Excel-sheets, en is gevalideerd in diverse HCI-contexten zoals mobiele gezondheidsapps en e-commerce, zoals beschreven in talrijke publicaties op de UEQ-website

Hoofdstuk 4: Resultaten

Hoofdstuk 5: Discussie

Hoofdstuk 6: Conclusie